Question 1: What is Ensemble Learning in machine learning? Explain the key idea behind it.
Ans:- **Ensemble Learning in Machine Learning**

Ensemble Learning is a machine learning technique in which **multiple models (called base learners or weak learners) are combined to solve the same problem and improve overall performance**.

Key Idea

The main idea behind ensemble learning is that **a group of models working together often produces better predictions than a single model**. Different models may make different errors, and by combining their outputs, the ensemble can reduce errors and increase accuracy, robustness, and generalization.

How It Works

1. Train multiple machine learning models.
2. Each model makes its own prediction.
3. Combine the predictions using methods such as:
    - **Voting** (for classification)
    - **Averaging** (for regression)
    - **Weighted combination** of predictions

Benefits of Ensemble Learning

- Higher prediction accuracy
- Reduced overfitting
- Improved stability and reliability
- Better handling of complex datasets

Common Ensemble Methods

1. **Bagging (Bootstrap Aggregating)**
    - Trains models on different random subsets of data.
    - Example: **Random Forest**.
2. **Boosting**

    - Models are trained sequentially, with each new model focusing on correcting previous errors.
    - Examples: **AdaBoost**, **Gradient Boosting**, **XGBoost**.
3. **Stacking**
    - Combines predictions from multiple models using another model (meta-learner).

Example

Suppose three classifiers predict whether an email is spam:

- Model A → Spam
- Model B → Spam
- Model C → Not Spam

Question 2: What is the difference between Bagging and Boosting?
Ans:-

Bagging (Bootstrap Aggregating)	Boosting
Models are trained **independently** and in parallel.	Models are trained **sequentially**, one after another.
Uses **random bootstrap samples** (random subsets with replacement) of the training data.	Uses the **entire dataset**, giving more weight to previously misclassified samples.
Reduces **variance** and helps prevent overfitting.	Reduces **bias** and improves prediction accuracy.
Each model has **equal importance** in the final prediction.	Later models have **greater focus** on correcting earlier mistakes.

Final prediction is made by **majority voting** (classification) or **averaging** (regression).	Final prediction is made using a **weighted combination** of all models.
Training is generally **faster** because models can run in parallel.	Training is generally **slower** because models depend on previous models.
Less sensitive to noisy data and outliers.	More sensitive to noisy data and outliers.
Ex:- Random Forest etc.	Ex:- AdaBoost,XGBoost GradientBoost

Question 3: What is bootstrap sampling and what role does it play in Bagging methods like Random Forest?

Ans:- **Bootstrap sampling** is a statistical technique in which **multiple random samples are created from the original dataset with replacement**. Because sampling is done with replacement, the same data point can appear multiple times in a sample, while some data points may not be selected.

Role of Bootstrap Sampling in Bagging (Random Forest)

Bootstrap sampling is the foundation of **Bagging (Bootstrap Aggregating)** and is used to create different training datasets for each model.

1. Multiple bootstrap samples are generated from the original dataset.
2. A separate decision tree is trained on each bootstrap sample.
3. Since each tree is trained on different data, the trees make different predictions.
4. The final prediction is obtained by:
    - **Majority voting** for classification problems.

- **Averaging** for regression problems.

Importance of Bootstrap Sampling

- Creates diversity among the models.
- Reduces overfitting by preventing all models from learning the same patterns.
- Reduces variance and improves the stability of the model.
- Increases the overall accuracy and robustness of the ensemble.

Example

Suppose the original dataset contains five records:

A, B, C, D, E

A bootstrap sample might be:

A, C, C, D, E

Here, **C** appears twice, while **B** is not selected. Different bootstrap samples are used to train different decision trees in a **Random Forest**.

Question 4: What are Out-of-Bag (OOB) samples and how is OOB score used to evaluate ensemble models?
Ans:- **Out-of-Bag (OOB) samples** are the data points from the original dataset that **are not included in a particular bootstrap sample** during the training of Bagging models such as **Random Forest**.

Since bootstrap sampling is done **with replacement**, about **63%** of the original data is used to train each decision tree, while the remaining **37%** is left out. These unused data points are called **Out-of-Bag (OOB) samples**.

How OOB Score Is Used

The OOB samples act as a **validation set** for the corresponding decision tree.

The process is:

1. Train each decision tree using a bootstrap sample.
2. Use the OOB samples (not used during training) to test that tree.
3. Repeat this for all trees in the forest.
4. Combine the predictions for each data point from all trees where it was an OOB sample.
5. Calculate the overall prediction accuracy, called the **OOB score**.

Advantages of OOB Score

- Provides an estimate of the model's performance without needing a separate test dataset.
- Saves time and computational resources.
- Helps detect overfitting.
- Gives a reliable estimate of the model's generalization accuracy.

Question 5: Compare feature importance analysis in a single Decision Tree vs. a Random Forest.

Ans:-

Single Decision Tree	Random Forest
Feature importance is calculated from **one decision tree** based on how much each feature reduces impurity (e.g., Gini Index or Entropy).	Feature importance is calculated by **averaging the importance scores across all trees** in the forest.
More sensitive to the training data and can change significantly with small changes in the dataset.	More stable and reliable because it combines the results of many trees.

Can be biased due to overfitting or the structure of a single tree.	Reduces overfitting and provides a more robust estimate of feature importance.
Easier to interpret since only one tree is involved.	Slightly less interpretable because importance comes from multiple trees, but generally more accurate.
Suitable for small datasets and simple problems.	Better suited for large and complex datasets with many features.

Key Differences

- **Single Decision Tree:** Feature importance is based on one tree, making it easy to understand but less reliable and more prone to overfitting.
- **Random Forest:** Feature importance is obtained by averaging the importance across many decision trees, resulting in more accurate, stable, and robust importance scores.

Question 6: Write a Python program to: ● Load the Breast Cancer dataset using sklearn.datasets.load_breast_cancer() ● Train a Random Forest Classifier ● Print the top 5 most important features based on feature importance scores. (Include your Python code and output in the code box below.)

Ans:-

```
# Import required libraries

from sklearn.datasets import load_breast_cancer
```

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

# Load Breast Cancer dataset
data = load_breast_cancer()

# Features and target
X = data.data
y = data.target
feature_names = data.feature_names

# Train Random Forest Classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

# Get feature importance scores
importances = rf.feature_importances_

# Create DataFrame for easy sorting
feature_importance = pd.DataFrame({
    'Feature': feature_names,
```

```
    'Importance': importances
})

# Sort by importance in descending order
feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

# Print top 5 most important features
print("Top 5 Most Important Features:")
print(feature_importance.head(5))
```

Output :-

Top 5 Most Important Features:

	Feature	Importance
22	worst perimeter	0.153
27	worst concave points	0.144
20	worst radius	0.118
7	mean concave points	0.104
23	worst area	0.091

Question 7: Write a Python program to:

● Train a Bagging Classifier using Decision Trees on the Iris dataset

● Evaluate its accuracy and compare with a single Decision Tree (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import BaggingClassifier

from sklearn.metrics import accuracy_score


# Load the Iris dataset

iris = load_iris()

X = iris.data

y = iris.target


# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# Train a single Decision Tree
```

```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

# Predict using Decision Tree
dt_predictions = dt_model.predict(X_test)

# Calculate Decision Tree accuracy
dt_accuracy = accuracy_score(y_test, dt_predictions)

# Train a Bagging Classifier with Decision Trees
bagging_model = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)
bagging_model.fit(X_train, y_train)

# Predict using Bagging Classifier
bagging_predictions = bagging_model.predict(X_test)

# Calculate Bagging accuracy
```

bagging_accuracy = accuracy_score(y_test, bagging_predictions)

Print results

print("Decision Tree Accuracy:", round(dt_accuracy, 4))

print("Bagging Classifier Accuracy:", round(bagging_accuracy, 4))

Output:- Decision Tree Accuracy: 1.0000

Bagging Classifier Accuracy: 1.0000

Question 8: Write a Python program to:

● Train a Random Forest Classifier

● Tune hyperparameters max_depth and n_estimators using GridSearchCV

● Print the best parameters and final accuracy (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

Load the Breast Cancer dataset

```
data = load_breast_cancer()
X = data.data
y = data.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create the Random Forest model
rf = RandomForestClassifier(random_state=42)

# Define the parameter grid
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [5, 10, 15, None]
}

# Perform Grid Search with 5-fold Cross Validation
grid_search = GridSearchCV(
    estimator=rf,
```

```
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
    n_jobs=-1
)

# Train the model
grid_search.fit(X_train, y_train)

# Best model
best_model = grid_search.best_estimator_

# Make predictions
y_pred = best_model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print results
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Score:", round(grid_search.best_score_, 4))
```

print("Test Accuracy:", round(accuracy, 4))

Output :- Best Parameters: {'max_depth': 10, 'n_estimators': 100}

Best Cross-Validation Score: 0.9648

Test Accuracy: 0.9649

Question 9: Write a Python program to:

● Train a Bagging Regressor and a Random Forest Regressor on the California Housing dataset

● Compare their Mean Squared Errors (MSE) (Include your Python code and output in the code box below.)

Ans:- # Import required libraries

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.ensemble import BaggingRegressor, RandomForestRegressor

from sklearn.tree import DecisionTreeRegressor

from sklearn.metrics import mean_squared_error

Load the California Housing dataset

housing = fetch_california_housing()

X = housing.data

y = housing.target

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Bagging Regressor
bagging_model = BaggingRegressor(
    estimator=DecisionTreeRegressor(),
    n_estimators=100,
    random_state=42
)
bagging_model.fit(X_train, y_train)

# Predict using Bagging Regressor
bagging_pred = bagging_model.predict(X_test)

# Calculate Bagging MSE
bagging_mse = mean_squared_error(y_test, bagging_pred)

# Train Random Forest Regressor
```

```
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)
rf_model.fit(X_train, y_train)

# Predict using Random Forest Regressor
rf_pred = rf_model.predict(X_test)

# Calculate Random Forest MSE
rf_mse = mean_squared_error(y_test, rf_pred)

# Print the results
print("Bagging Regressor MSE:", round(bagging_mse, 4))
print("Random Forest Regressor MSE:", round(rf_mse, 4))
```

Output:- Bagging Regressor MSE: 0.2524

Random Forest Regressor MSE: 0.2554

Question 10: You are working as a data scientist at a financial institution to predict loan default. You have access to customer demographic and transaction history data. You decide to use ensemble techniques to increase model performance. Explain your step-by-step approach to:

● Choose between Bagging or Boosting

● Handle overfitting

● Select base models

● Evaluate performance using cross-validation

● Justify how ensemble learning improves decision-making in this real-world context.

Ans:- **Step 1: Choose Between Bagging and Boosting**

- **Bagging** (e.g., Random Forest) is suitable when the main issue is **high variance** and overfitting. It builds multiple decision trees on different bootstrap samples and combines their predictions.
- **Boosting** (e.g., Gradient Boosting, AdaBoost, XGBoost) is suitable when the goal is to **maximize prediction accuracy** by correcting the errors made by previous models.
- **Choice:** For loan default prediction, I would choose **Boosting** because financial institutions require high predictive accuracy to identify risky customers. If the dataset is noisy or overfitting is a concern, **Random Forest** is also an excellent choice.

Step 2: Handle Overfitting

To reduce overfitting, I would:

- Split the dataset into **training, validation, and testing** sets.
- Use **cross-validation** to evaluate model stability.
- Tune hyperparameters such as:
  - `max_depth`
  - `n_estimators`
  - `min_samples_split`
  - `learning_rate` (for Boosting)
- Apply **early stopping** for boosting algorithms.
- Remove irrelevant features and handle missing values appropriately.

Step 3: Select Base Models

I would use **Decision Trees** as the base learners because:

- They can model complex, non-linear relationships.
- They work well with both numerical and categorical features.
- They require minimal data preprocessing.
- They are the standard base estimator for Bagging and Boosting algorithms.

Step 4: Evaluate Performance Using Cross-Validation

I would use **5-fold or 10-fold Cross-Validation**:

1. Divide the dataset into equal folds.
2. Train the model on k−1k-1k−1 folds.
3. Test it on the remaining fold.
4. Repeat until every fold has been used as the test set.
5. Compute the average performance.

Evaluation metrics include:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC Score

Since loan default prediction is often an **imbalanced classification problem**, I would focus more on **Precision, Recall, F1-score, and ROC-AUC** rather than accuracy alone.

Step 5: Justification of Ensemble Learning

Ensemble learning improves decision-making because:

- It combines predictions from multiple models, resulting in **higher accuracy**.
- It reduces **overfitting** and improves generalization.

- It handles complex relationships in customer demographic and transaction data effectively.
- It is more robust to noisy or incomplete data.
- It reduces prediction errors compared to a single model.